

Orquestación de un pipeline de datos abiertos con un solo script: el caso de los siniestros viales registrados por Carabineros de Chile

Rodrigo Sena

Ingeniero Civil en Geografía, Universidad de Santiago de Chile
Magíster en Data Science, Pontificia Universidad Católica de Chile
Gerente General, SEDRA Ingeniería Ltda., Santiago, Chile

Octubre de 2026

Resumen

Los sistemas de orquestación de flujos de datos, como Airflow o Snakemake, resuelven la ejecución ordenada, el reinicio parcial y la trazabilidad, pero exigen una infraestructura que una consultora pequeña rara vez mantiene. Este artículo relata la orquestación de un pipeline de nueve etapas que consolida y georreferencia 1.275.080 siniestros de tránsito registrados por Carabineros de Chile entre 2010 y 2026, implementado en un solo archivo de Python y publicado en servicios gratuitos. Se describen seis mecanismos: contratos por archivo con ejecución selectiva de etapas, invalidación explícita de cachés, escritura atómica con tolerancia a fallos, deduplicación del cómputo, observabilidad mediante bitácoras con muestras reproducibles y un ciclo de corrección manual integrado al flujo. La corrida completa tarda 23 minutos en un computador personal y rehacer solo la geocodificación, 4 minutos y 20 segundos, lo que permitió 18 iteraciones de las reglas de dirección en cuatro días. En la publicación, los límites de la plataforma obligaron a partir los datos en archivos de menos de 23 MB y a reducir la memoria del tablero de 3.287 MB a 743 MB. Se discute cuándo un script único es suficiente y cuándo conviene migrar a un orquestador.

Palabras clave: *orquestación de datos; pipelines ETL; reproducibilidad; geocodificación; Streamlit; siniestros de tránsito*

Abstract

Workflow orchestrators such as Airflow or Snakemake provide ordered execution, partial reruns and provenance, but require infrastructure that small consultancies seldom maintain. This paper reports the orchestration of a nine-stage, single-file Python pipeline that consolidates and geocodes 1,275,080 police-recorded road crashes in Chile (2010 to 2026) and publishes them on free services. Six mechanisms are described: file contracts with selective stage execution, explicit cache invalidation, atomic writes with fault tolerance, deduplicated computation, log-based observability with reproducible samples, and a built-in manual correction loop. A full run takes 23 minutes on a laptop and rerunning geocoding alone takes 4 min 20 s, which enabled 18 iterations of the address rules in four days. Platform limits required splitting the data into files under 23 MB and cutting dashboard memory from 3,287 MB to 743 MB.

Keywords: *data orchestration; ETL pipelines; reproducibility; geocoding; Streamlit; road crashes*

1. Introducción

Un pipeline de datos no es solo una secuencia de transformaciones. Necesita decidir qué ejecutar y en qué orden, qué resultados reutilizar, qué hacer cuando una pieza falla y cómo dejar constancia de lo que hizo. A ese conjunto de decisiones se le llama orquestación, y las herramientas que lo resuelven, desde Make hasta Airflow o Snakemake, son parte habitual de la ingeniería de datos.

Esas herramientas suponen, sin embargo, un entorno que no siempre existe. En una consultora de ingeniería de tránsito, el pipeline se ejecuta en el computador personal de un profesional, en Windows y dentro de una carpeta sincronizada con OneDrive, y sus resultados se publican en un sitio web administrado con WordPress. Instalar y mantener un servidor de orquestación tiene en ese contexto un costo desproporcionado frente al problema.

Este artículo relata cómo se orquestó, en esas condiciones, un pipeline que transforma los registros de siniestros de tránsito de Carabineros de Chile en una base georreferenciada y en un tablero público. El sistema es un único archivo de Python, `script.py`, de 3.549 líneas, más un segundo archivo para el tablero. Las contribuciones son tres: la descripción de seis mecanismos de orquestación implementados sin dependencias de infraestructura, la medición de su efecto en el ciclo de desarrollo y la descripción de la orquestación del despliegue en servicios gratuitos, incluido el diagnóstico de una falla de memoria que la plataforma no informaba.

2. Trabajos relacionados

Make introdujo la idea de describir un proceso como reglas con dependencias y de rehacer solo lo que quedó desactualizado, comparando las fechas de modificación de los archivos [1]. Los sistemas de flujos de trabajo científicos generalizaron ese enfoque con descripción declarativa de tareas, planificación, tolerancia a fallos y registro de procedencia, como resume la revisión de Deelman et al. [2]. Snakemake lleva las reglas de Make a Python y agrega entornos reproducibles y ejecución en clústeres [3], y su evolución posterior pone el acento en la sostenibilidad de los análisis, es decir, en que sigan siendo legibles y ejecutables con el tiempo [4]. Airflow, orientado a procesos periódicos de empresa, modela cada flujo como un grafo dirigido acíclico de tareas con un planificador y una interfaz de monitoreo [5].

En paralelo, la literatura sobre reproducibilidad computacional ha propuesto reglas que no dependen de una herramienta concreta: registrar cómo se produjo cada resultado, evitar pasos manuales de manipulación de datos, guardar los resultados intermedios y fijar las semillas de los procesos aleatorios [6], así como conservar los datos originales y registrar cada paso de procesamiento [7]. Este trabajo adopta esas reglas como requisitos y muestra que, en un pipeline lineal de tamaño moderado, pueden cumplirse dentro del propio script.

3. El pipeline

Carabineros publica los siniestros del formulario OS2 como planillas Excel por año y por tabla (siniestros, personas y vehículos), con planillas mensuales para el año en curso [8]. En la corrida del 6 de octubre de 2026 se procesaron 72 planillas, 24 por tabla. El formato cambia entre años, la codificación de las planillas antiguas está dañada y la ubicación del siniestro es texto libre repartido en cinco campos, sin coordenadas.

El pipeline tiene nueve etapas (Fig. 1). Las seis primeras descargan, convierten, traducen a un esquema fijo, normalizan, resumen y unen los archivos en tres consolidados. La séptima reescribe la ubicación en una dirección estándar y la octava la geocodifica contra un índice local de OpenStreetMap, distribuido bajo licencia ODbL [9], complementado con la red vial de la Dirección de Vialidad, que publica el kilometraje de cada ruta. La novena exporta una tabla compacta para el tablero, una aplicación de Streamlit. Las reglas de dirección y de geocodificación y sus resultados se analizan en [10], y cada función del código se documenta en [11].

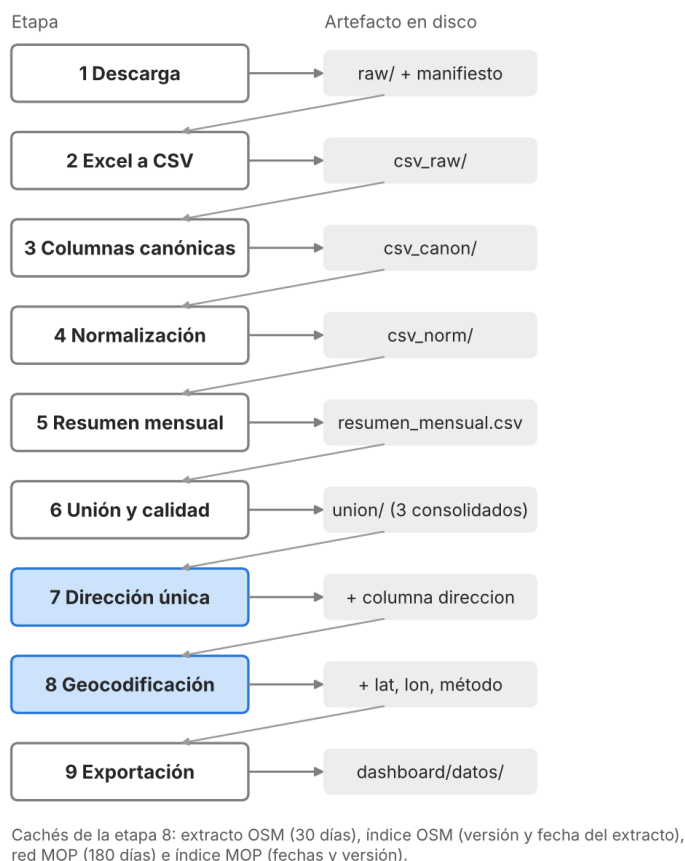


Figura 1. Etapas del pipeline y artefacto que cada una deja en disco. Cada etapa lee el artefacto de la anterior; las etapas 7 y 8 agregan columnas al consolidado de siniestros.

4. Mecanismos de orquestación

4.1 Contratos por archivo y ejecución selectiva

Cada etapa está envuelta en un bloque `if DESDE <= N <= HASTA`; y se comunica con las demás solo a través de archivos con nombre y formato fijos: CSV separados por punto y coma, en UTF-8 con marca de orden de bytes y con todas las columnas leídas como texto. Ese contrato hace que el grafo de dependencias sea implícito y lineal, y que cualquier tramo pueda ejecutarse por separado con `--desde` y `--hasta`, porque una etapa no ejecutada no toca sus archivos y la siguiente lee lo que haya en disco.

A diferencia de Make, el script no deduce qué etapas están desactualizadas: el usuario elige desde dónde rehacer. La decisión es deliberada, porque las causas de rehacer son casi siempre cambios en las reglas del código y no en los archivos, y Make no las vería. En la práctica bastan tres puntos de reinicio: `--desde 7` tras cambiar reglas de dirección, `--desde 8` tras agregar alias de calles y `--desde 9` tras cambiar la exportación.

4.2 Invalidación explícita de cachés

Dos insumos son costosos de obtener y cambian poco: el extracto de OpenStreetMap de Chile y la red vial del MOP. Ambos se guardan en disco junto con índices derivados, y cada caché lleva una regla de invalidación explícita. El extracto se renueva si tiene más de 30 días y la red, si tiene más de 180. El índice de OpenStreetMap guarda la fecha de modificación del extracto y un número de versión de su estructura (`OSM_INDICE_VERSION`), y se reconstruye si cualquiera de los dos cambia; el índice del MOP usa como clave las fechas de ambos insumos y la misma versión. La opción `--osm-actualizar` fuerza todo, y `--skip-existing` omite las descargas y conversiones cuyo resultado ya existe y es más reciente que su origen.

4.3 Escritura atómica y tolerancia a fallos

Todo archivo principal se escribe primero como temporal y luego reemplaza al definitivo con `os.replace`, que es atómico. En una carpeta sincronizada con OneDrive, el reemplazo falla a veces porque el servicio o Excel mantienen el archivo abierto, por lo que la función reintenta seis veces con cinco

segundos de espera antes de pedir al usuario que cierre el archivo. La exportación del tablero aplica la misma idea a un conjunto de archivos: escribe todas las partes nuevas y solo entonces borra las anteriores.

Las fallas se aíslan al nivel más pequeño que tenga sentido. En las etapas 2 a 5 un archivo con error se registra y la etapa continúa con los demás; en la geocodificación, una consulta con error queda sin coordenadas sin detener las cerca de 500.000 restantes; y si la red del MOP no está disponible, los kilómetros se ubican solo con los hitos de OpenStreetMap. Antes de las etapas 8 y 9 el script verifica las bibliotecas opcionales y, si falta alguna, se detiene con la orden exacta para instalarla en el intérprete que se está usando. Al final imprime un resumen por etapa con archivos correctos, omitidos y fallidos, y termina con código de salida 1 si hubo alguna falla.

4.4 Deduplicación del cómputo

Las transformaciones costosas se aplican una sola vez por valor o combinación distinta y el resultado se vuelve a unir a las filas. La normalización de texto se evalúa por valor único de cada columna; la construcción de direcciones reduce 1.275.080 filas a 582.245 combinaciones de componentes; y la geocodificación, a 497.143 consultas distintas de dirección y comuna. Las funciones de limpieza de nombres se memorizan con `functools.lru_cache`, y el geocodificador guarda en memoria los nombres ya resueltos y las geometrías ya combinadas.

4.5 Observabilidad

El pipeline escribe una bitácora general y dos bitácoras propias para las etapas más delicadas, la construcción de direcciones y la geocodificación. Además de cifras de cobertura, cada regla de limpieza registra cuántas filas tocó y cinco ejemplos antes y después, y cada paso aproximado de coincidencia de nombres registra muestras aleatorias para buscar falsos positivos. La semilla de las muestras cambia en cada corrida y queda registrada, de modo que `--semilla N` reproduce exactamente una revisión anterior.

4.6 Corrección manual dentro del flujo

Una parte del trabajo no puede automatizarse: decidir a qué calle de OpenStreetMap corresponde un nombre mal escrito que ninguna regla resolvió. En lugar de hacerlo fuera del pipeline, que rompería la reproducibilidad, el script lo integra como un insumo. Cada corrida escribe las 500 calles sin pareja que suman más registros con sugerencias de nombres; el usuario completa las que quiera en una planilla, y la corrida siguiente copia esas filas a un archivo permanente de alias y las aplica con prioridad. La corrección manual queda así versionada como dato y se repite igual en cada ejecución.

Tabla 1. Mecanismos de orquestación y su implementación en el script

Mecanismo	Implementación	Efecto
Ejecución selectiva	--desde, --hasta y contratos por archivo	Reinicio desde cualquier etapa
Cachés	Antigüedad máxima, versión y fechas como clave	Índices reutilizados sin quedar obsoletos
Escritura atómica	Temporal y <code>os.replace</code> con reintentos	Archivos nunca escritos a medias
Aislamiento de fallas	Error por archivo o por consulta	Una falla no detiene la corrida
Deduplicación	Cálculo por valor o combinación única	Millones de filas, cientos de miles de casos
Observabilidad	Tres bitácoras, muestras con semilla	Cada regla auditada y repetible
Corrección manual	Alias sugeridos que pasan a ser insumo	Lo manual queda versionado como dato

5. Orquestación del despliegue

El tablero se publica en tres servicios encadenados (Fig. 2). El código y los datos exportados viven en un repositorio público de GitHub; Streamlit Community Cloud clona el repositorio, instala las dependencias y ejecuta la aplicación; y una página del sitio WordPress de la empresa la muestra dentro de un marco con el parámetro `?embed=true`, que oculta la interfaz propia de Streamlit [12]. Cada confirmación en el repositorio redespiega la aplicación, de modo que actualizar los datos consiste en ejecutar el pipeline y subir la carpeta exportada.

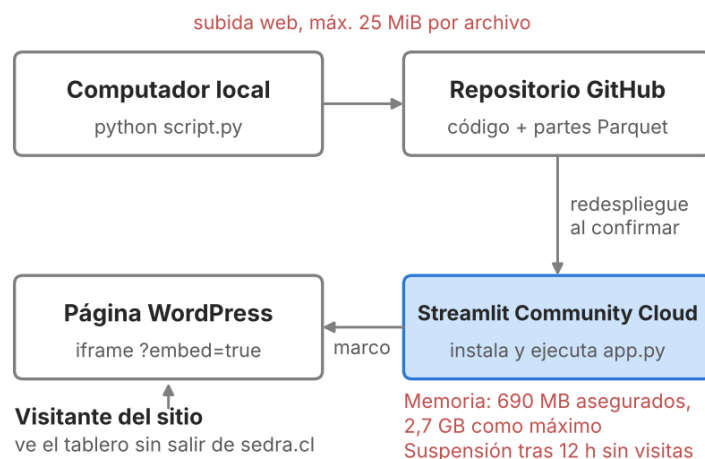


Figura 2. Cadena de despliegue y límites de cada servicio que condicionaron el diseño.

Los límites de cada servicio terminaron siendo parte de la orquestación. La interfaz web de GitHub no acepta archivos de más de 25 MiB [13], y el primer archivo exportado pesaba 26 MB; la exportación pasó a escribir partes numeradas de menos de 23 MB con compresión zstd de nivel 19, y el tablero lee y une todas las partes que encuentra. La plataforma elige el archivo de dependencias de la carpeta del programa principal antes que el de la raíz, y en la bitácora de instalación se observó que reemplazaba por su cuenta una versión de pyarrow con una falla conocida, lo que confirma que el entorno de ejecución no es del todo controlable desde el repositorio.

El límite más difícil fue la memoria. Las aplicaciones gratuitas tienen asegurados unos 690 MB y pueden llegar a 2,7 GB [14]. La primera versión publicada se detenía cada pocos minutos, y su bitácora no mostraba errores de Python, solo reinicios. La causa apareció al reproducir el uso con los datos reales en un entorno local: tras seis visitas, el proceso ocupaba 3.287 MB (Fig. 3). La tabla se guardaba con un caché que entrega una copia nueva en cada interacción [15], el filtro copiaba la tabla completa aun sin filtros activos y las tablas intermedias de cada interacción seguían vivas en la siguiente. Compartir una sola tabla entre sesiones, evitar las copias innecesarias y liberar las tablas intermedias al final de cada ejecución estabilizó el consumo en 743 MB.

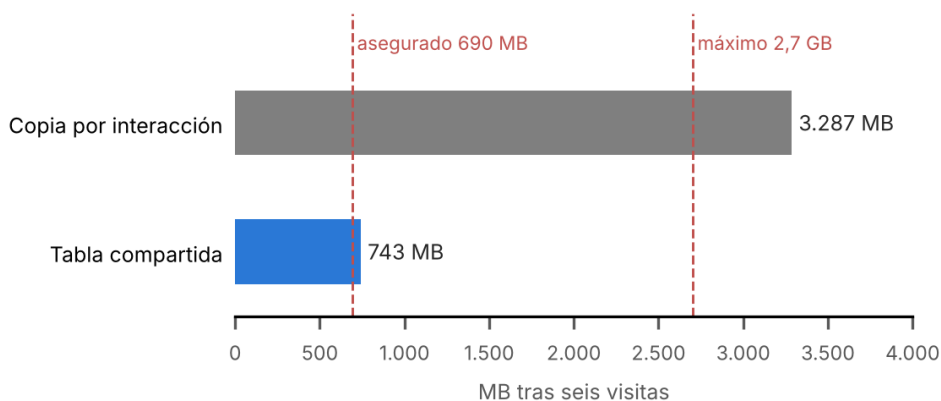


Figura 3. Memoria del tablero tras seis visitas con los datos reales, antes y después de la corrección, frente a los límites de la plataforma.

6. Evaluación

La corrida completa del 6 de octubre de 2026 tardó 23 minutos y 1 segundo sumando las etapas (Fig. 4). La conversión de Excel a CSV es la más lenta, con el 38,7 % del total, porque la lectura de planillas es costosa y se hace una vez por archivo; la descarga tardó 7 segundos. Las etapas que concentran las reglas, dirección y geocodificación, suman 5 minutos y 31 segundos.

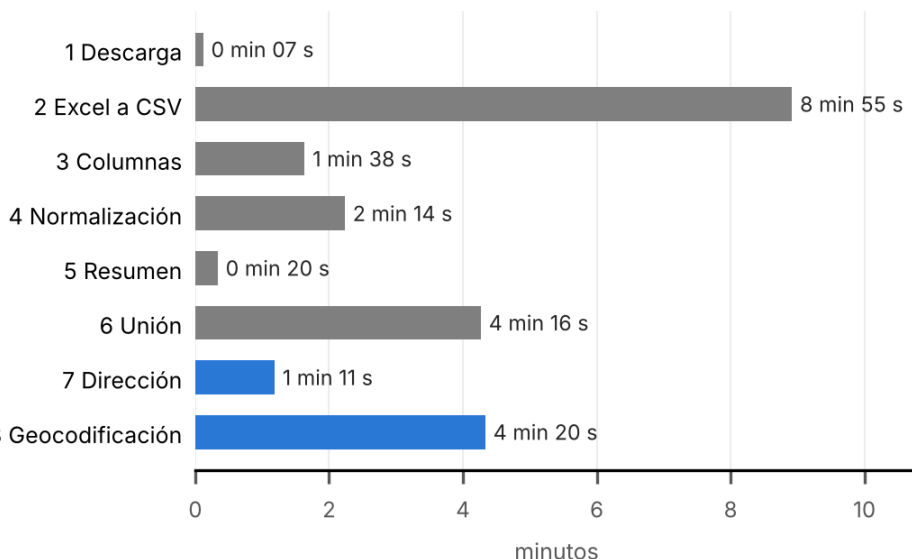


Figura 4. Duración de cada etapa en la corrida del 6 de octubre de 2026 en un computador personal. En azul, las etapas que se rehacen con más frecuencia durante el desarrollo.

El efecto de la ejecución selectiva se aprecia en el costo de rehacer desde cada punto (Tabla 2). Rehacer solo la geocodificación cuesta el 18,8 % de una corrida completa, y rehacer dirección y geocodificación, el 24,0 %. Entre el 3 y el 6 de octubre de 2026, las bitácoras registran 18 ejecuciones de la etapa de direcciones y 13 de la geocodificación; a costo de corrida completa habrían requerido cerca de siete horas solo de espera.

Tabla 2. Costo de rehacer el pipeline desde cada etapa, a partir de las duraciones de la Fig. 4

Punto de reinicio	Duración	% de la corrida completa
--desde 8	4 min 20 s	18,8
--desde 7	5 min 31 s	24,0
--desde 6	9 min 47 s	42,5
--desde 4	12 min 21 s	53,7
Corrida completa	23 min 01 s	100,0

En términos de resultado, el pipeline consolidó 1.275.080 siniestros, 2.583.141 personas y 2.257.681 vehículos, y el 91,6 % de los siniestros obtuvo coordenadas; el 52,2 % del total quedó ubicado con una precisión estimada de 60 m o mejor. La invalidación de cachés funcionó como se esperaba: en las corridas en que no cambió el extracto, los índices se leyeron de disco, y la reconstrucción del índice de OpenStreetMap, cuando fue necesaria, agregó cerca de 4 minutos.

7. Discusión

El caso muestra que los requisitos de reproducibilidad de la literatura [6], [7] pueden cumplirse sin un orquestador cuando el flujo es lineal, se ejecuta en una sola máquina y lo mantiene una sola persona. La ejecución selectiva por número de etapa reemplaza al cálculo automático de dependencias de Make [1] con una regla que el usuario entiende, y las cachés con claves explícitas cubren el caso que Make no ve: un cambio en el código que vuelve obsoleto un resultado.

El costo de esa simplicidad también es claro. Un archivo de 3.549 líneas es difícil de revisar, las etapas no se ejecutan en paralelo aunque algunas podrían hacerlo, la decisión de desde dónde rehacer depende de que el usuario conozca el sistema y no hay pruebas automáticas de las reglas. Un orquestador como Snakemake [3], [4] o Airflow [5] se justificaría si el proceso tuviera que ejecutarse con calendario, lo mantuvieran varias personas o se ramificara en flujos independientes. Los contratos por archivo facilitan esa migración, porque cada etapa ya es una tarea con entradas y salidas definidas.

En el despliegue, la lección principal es que una plataforma gestionada oculta causas. El reinicio sin error solo se explicó al medir la memoria con los datos reales, y la solución exigió entender el modelo de ejecución de Streamlit, que vuelve a correr el script completo en cada interacción. Las cifras de este trabajo provienen de una sola máquina y de una sola corrida por configuración, por lo que deben leerse como órdenes de magnitud.

8. Conclusiones

Se orquestó un pipeline de nueve etapas que georreferencia 1,3 millones de siniestros de tránsito con un solo archivo de Python, sin servidores propios y con publicación en servicios gratuitos. Seis mecanismos sencillos, contratos por archivo con ejecución selectiva, cachés con invalidación explícita, escritura atómica, aislamiento de fallas, deduplicación y bitácoras con muestras reproducibles, más un ciclo de corrección manual integrado como dato, bastaron para cumplir los requisitos habituales de reproducibilidad y para sostener un desarrollo de muchas iteraciones cortas. En la publicación, los límites de tamaño y de memoria de la plataforma obligaron a cambios de diseño que conviene prever desde el inicio.

Referencias

- [1] S. I. Feldman, «Make: a program for maintaining computer programs», *Software: Practice and Experience*, vol. 9, n.º 4, pp. 255–265, 1979. <https://doi.org/10.1002/spe.4380090402>
- [2] E. Deelman, D. Gannon, M. Shields e I. Taylor, «Workflows and e-Science: an overview of workflow system features and capabilities», *Future Generation Computer Systems*, vol. 25, n.º 5, pp. 528–540, 2009. <https://doi.org/10.1016/j.future.2008.06.012>
- [3] J. Köster y S. Rahmann, «Snakemake: a scalable bioinformatics workflow engine», *Bioinformatics*, vol. 28, n.º 19, pp. 2520–2522, 2012. <https://doi.org/10.1093/bioinformatics/bts480>
- [4] F. Mölder, K. P. Jablonski, B. Letcher, M. B. Hall, C. H. Tomkins-Tinch, V. Sochat et al., «Sustainable data analysis with Snakemake», *F1000Research*, vol. 10, art. 33, 2021. <https://doi.org/10.12688/f1000research.29032.2>
- [5] Apache Software Foundation, «Dags», documentación de Apache Airflow. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>
- [6] G. K. Sandve, A. Nekrutenko, J. Taylor y E. Hovig, «Ten simple rules for reproducible computational research», *PLOS Computational Biology*, vol. 9, n.º 10, e1003285, 2013. <https://doi.org/10.1371/journal.pcbi.1003285>
- [7] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt y T. K. Teal, «Good enough practices in scientific computing», *PLOS Computational Biology*, vol. 13, n.º 6, e1005510, 2017. <https://doi.org/10.1371/journal.pcbi.1005510>
- [8] Carabineros de Chile, «Estadísticas de siniestros de tránsito, Departamento OS2», transparencia activa. https://www.carabineros.cl/transparencia/tproactiva/rpro_os2.html
- [9] OpenStreetMap Foundation, «Copyright and License». <https://www.openstreetmap.org/copyright>
- [10] R. Sena, «Desarrollo de un sistema reproducible de georreferenciación de siniestros viales en Chile», Anexo A, SEDRA Ingeniería Ltda., Santiago, 2026.
- [11] R. Sena, «Documentación técnica del pipeline OS2 de siniestros viales», versión 2, y «Referencia de funciones», SEDRA Ingeniería Ltda., Santiago, 2026. Código: <https://github.com/arrozxf/siniestros-viales>
- [12] Streamlit, «Embed your app». <https://docs.streamlit.io/deploy/streamlit-community-cloud/share-your-app/embed-your-app>
- [13] GitHub, «About large files on GitHub». <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github>
- [14] Streamlit, «Manage your app: app resources and limits». <https://docs.streamlit.io/deploy/streamlit-community-cloud/manage-your-app>

[15] Streamlit, «Caching overview». <https://docs.streamlit.io/develop/concepts/architecture/caching>

Disponibilidad de datos y código. El código está en el repositorio público citado en [11]. Las duraciones y conteos provienen de las bitácoras de las corridas del 3 al 7 de octubre de 2026. Las coordenadas derivadas usan datos © colaboradores de OpenStreetMap, licencia ODbL.